

Distributed Systems

Fault tolerance and Consistency

Pedro García López

pgarcia@etse.urv.es/

Copyright

- © University Rovira i Virgili
- Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.1 or any later version published by the Free Software Foundation; provided its original author is mentioned and the link to <http://libre.act-europe.fr/> is kept at the bottom of every non-title slide. A copy of the license is available at:
<http://www.fsf.org/licenses/fdl.html>

Challenge: Coordination

- The solution to availability and scalability is to decentralize and replicate functions and data...but how do we coordinate the nodes?
 - data consistency
 - update propagation
 - mutual exclusion
 - consistent global states
 - group membership
 - group communication
 - event ordering
 - distributed consensus
 - quorum consensus

Overview

- Fault Tolerance
- The fundamental challenge of consensus
- General impossibility of consensus
 - in asynchronous networks (e.g., the Internet)
 - safety vs. liveness: “CAP theorem”
- Consistency models
- Transactions and 2PC

Fault Tolerance Basic Concepts

- Being fault tolerant is strongly related to what are called dependable systems
- Dependability implies the following:
 - 1.Availability
 - 2.Reliability
 - 3.Safety
 - 4.Maintainability

Failure Models

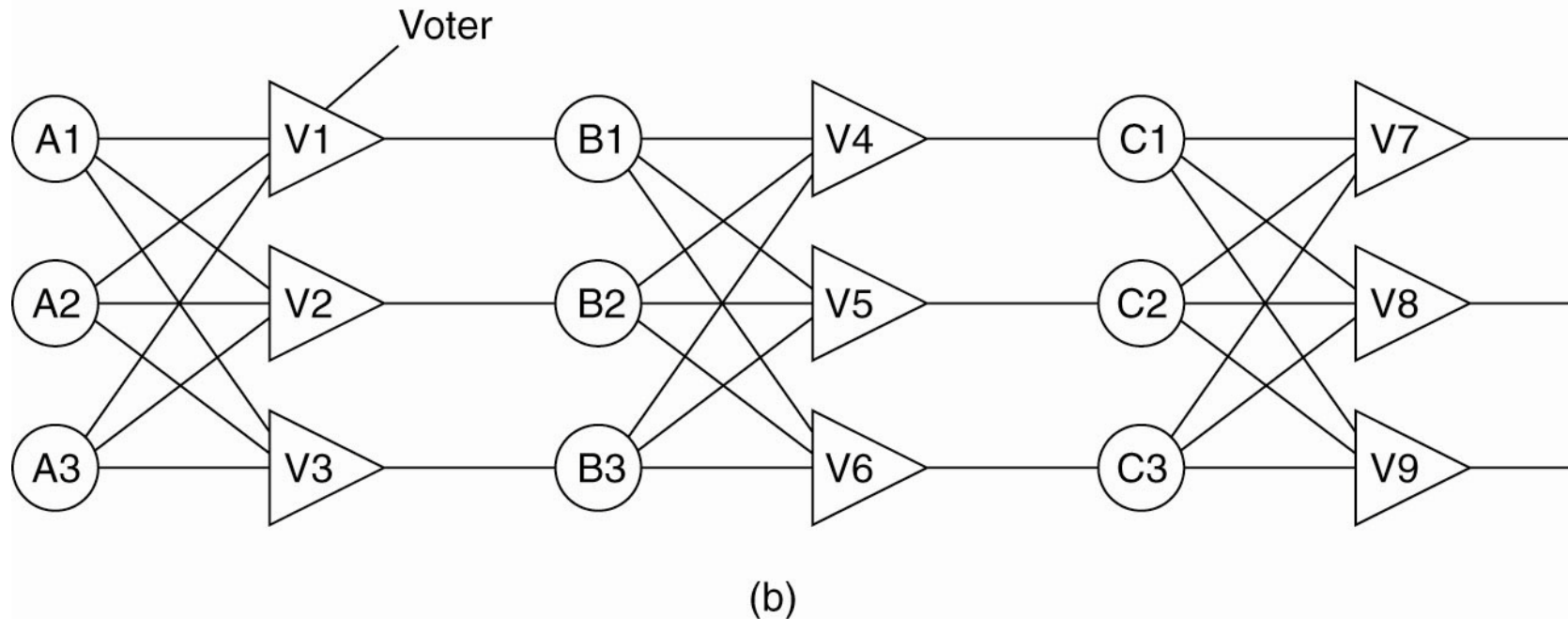
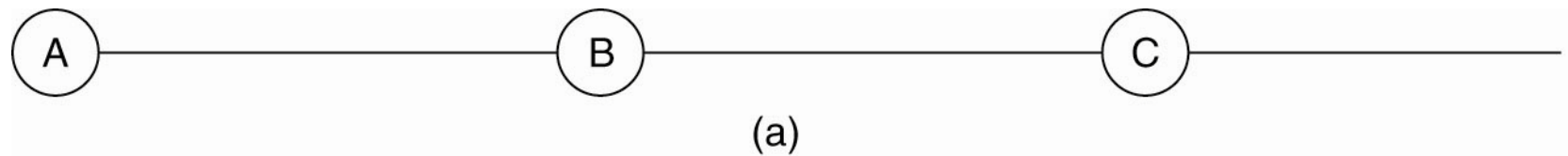
Type of failure	Description
Crash failure	A server halts, but is working correctly until it halts
Omission failure <i>Receive omission</i> <i>Send omission</i>	A server fails to respond to incoming requests A server fails to receive incoming messages A server fails to send messages
Timing failure	A server's response lies outside the specified time interval
Response failure <i>Value failure</i> <i>State transition failure</i>	A server's response is incorrect The value of the response is wrong The server deviates from the correct flow of control
Arbitrary failure	A server may produce arbitrary responses at arbitrary times

Node failures

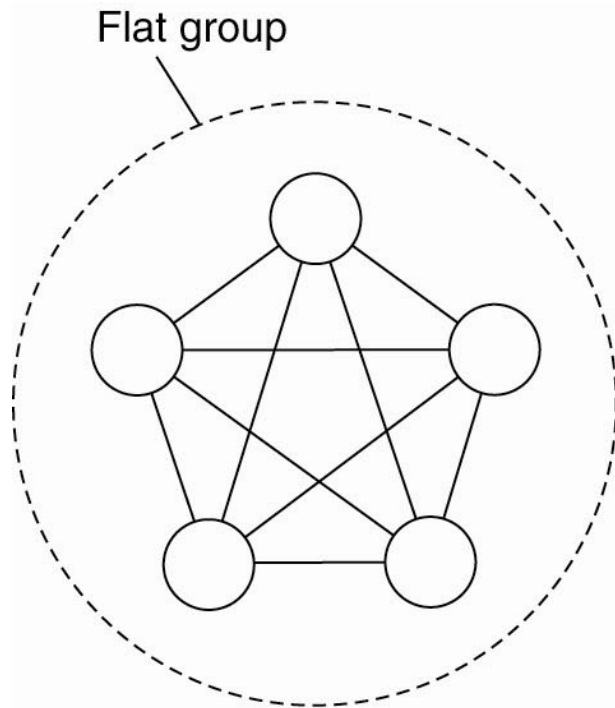
- **Fail-stop.** Nodes/actors may fail by stopping.
- **Byzantine.** Nodes/actors may fail without stopping.
 - Arbitrary, erratic, unexpected behavior
 - May be malicious and disruptive
- **Unfaithful** behavior
 - Actors may behave unfaithfully from self-interest.
 - If it is rational, is it Byzantine?
 - If it is rational, then it is expected.
 - If it is expected, then we can control it.
 - Design in incentives for faithful behavior, or disincentives for unfaithful behavior.

Failure Masking by Redundancy

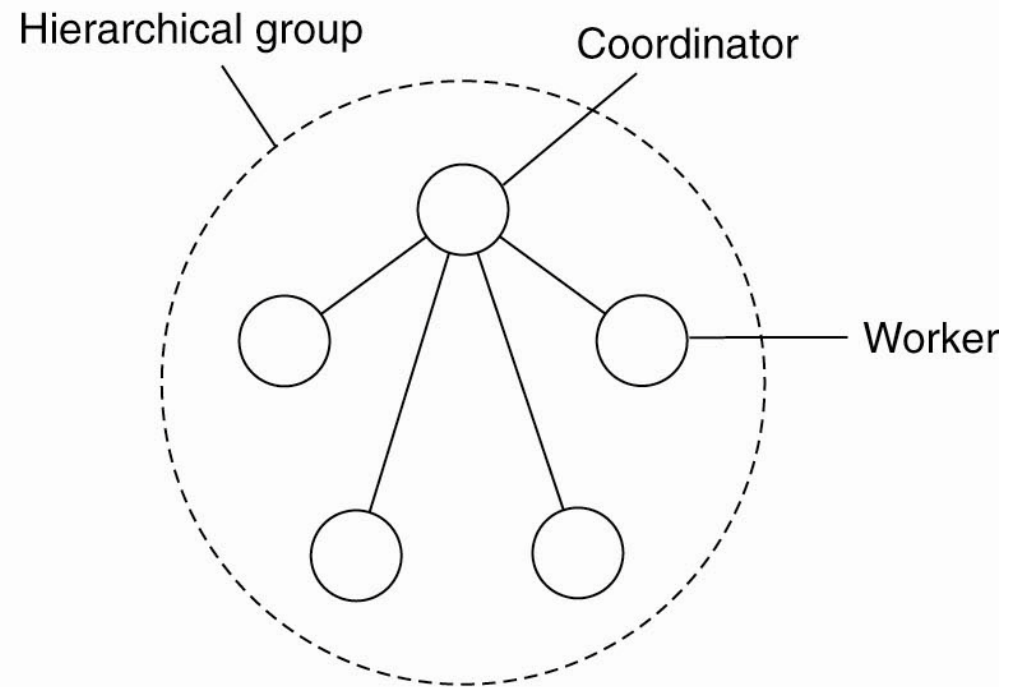
- Triple modular redundancy.



Flat Groups versus Hierarchical Groups



(a)



(b)

Properties of nodes

- Essential properties typically assumed by model:
 - Private state
 - Distributed memory: model sharing as messages
 - Executes a sequence of state transitions
 - Some transitions are reactions to messages
 - May have internal concurrency, but hide that
 - Unique identity
 - Local clocks with bounded drift

Node recovery

- Fail-stopped nodes may revive/restart.
 - Retain identity
 - Lose messages sent to them while failed
 - Arbitrary time to restart...or maybe never
- Restarted node may recover state at time of failure.
 - Lose state in volatile (primary) memory.
 - Restore state in non-volatile (secondary) memory.
 - Writes to non-volatile memory are expensive.
 - Design problem: recover complete states reliably, with minimal write cost.

Distributed System Models

- Synchronous model
 - Message delay is bounded and the bound is known.
 - E.g., delivery before next tick of a global clock.
 - Simplifies distributed algorithms
 - “learn just by watching the clock”
 - absence of a message conveys information.
- Asynchronous model
 - Message delays are finite, but unbounded/unknown
 - More realistic/general than synchronous model.
 - “Beware of any model with stronger assumptions.” - Burrows
 - Strictly harder/weaker than synchronous model.
 - Consensus is not always possible

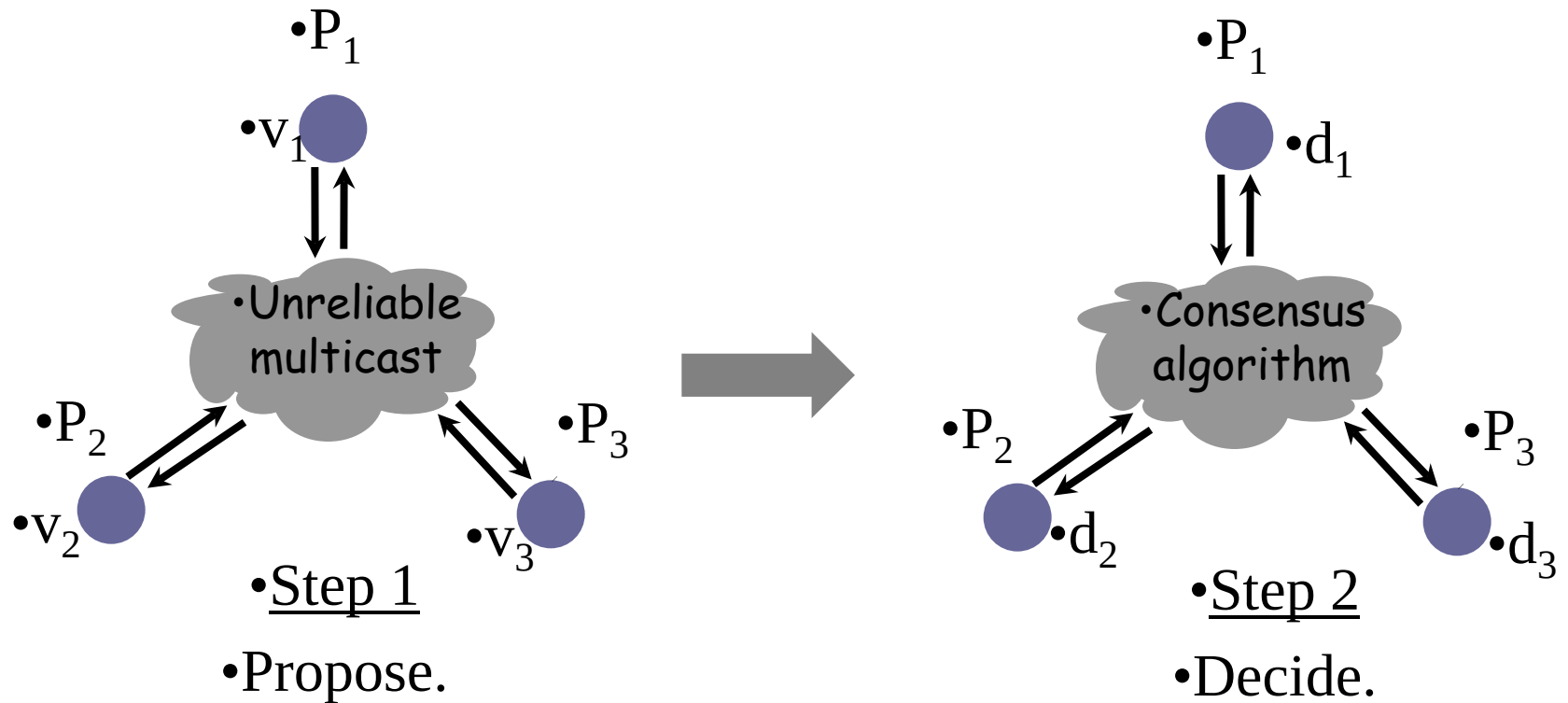
Messaging properties

- Other possible properties of the messaging model:
 - Messages may be lost.
 - Messages may be delivered out of order.
 - Messages may be duplicated.
- Do we need to consider these in our distributed system model?
- Or, can we solve them within the asynchronous model, without affecting its foundational properties?

Agreement in Faulty Systems

- Possible cases:
 1. Synchronous versus asynchronous systems.
 2. Communication delay is bounded or not.
 3. Message delivery is ordered or not.
 4. Message transmission is done through unicasting or multicasting.

Consensus



•Generalizes to N nodes/processes.

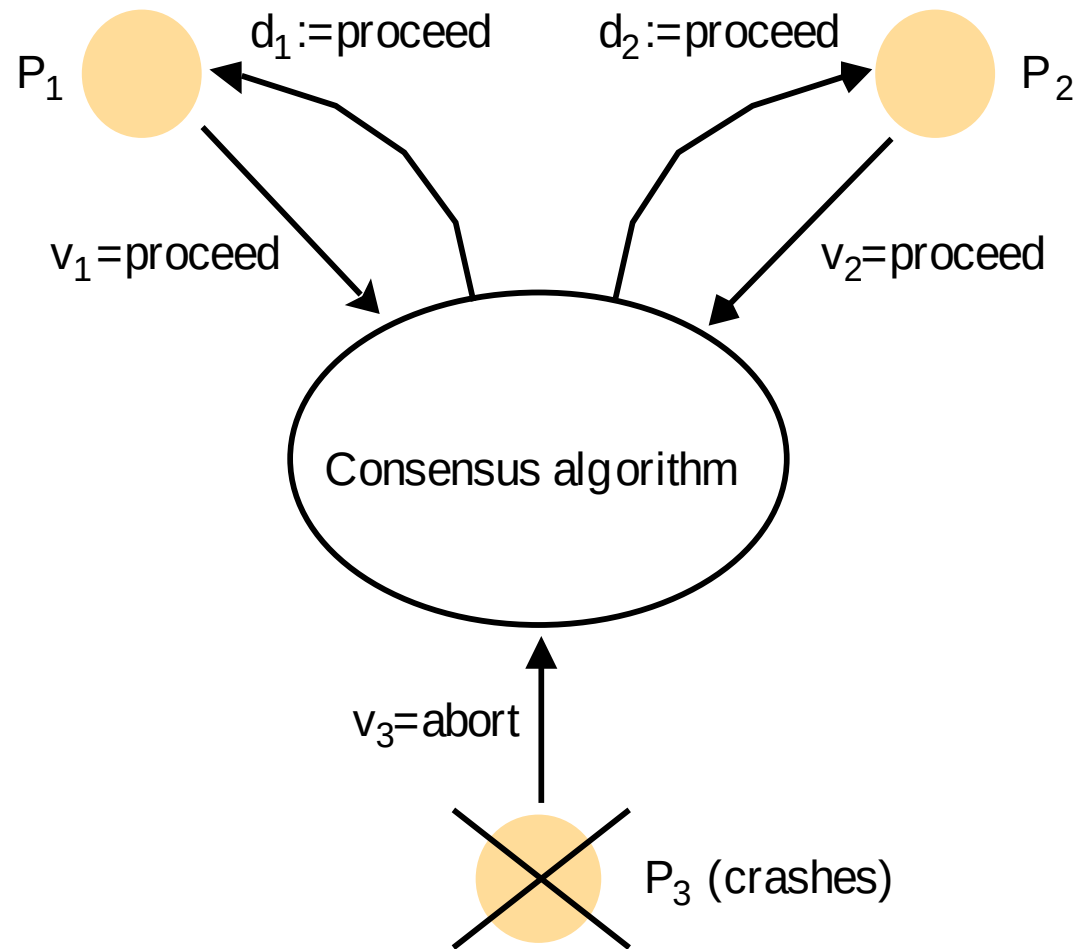
Properties for Correct Consensus

- Termination: All correct processes **eventually** decide.
- Agreement: All correct processes select the same d_i .
 - Or...(stronger) all processes that do decide select the same d_i , even if they later fail.
 - Called uniform consensus: “Uniform consensus is harder than consensus.”
- Integrity: All deciding processes select the “right” value.
 - As specified for the variants of the consensus problem.

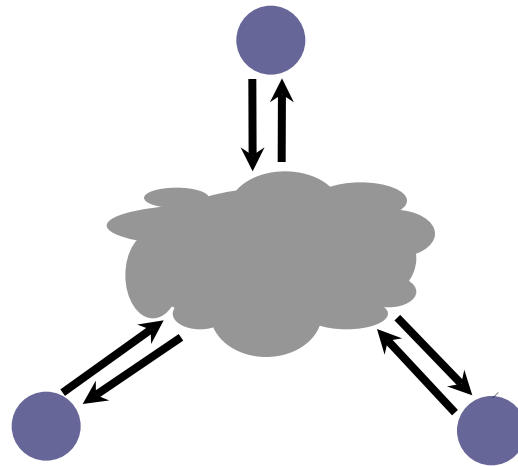
Properties of Distributed Algorithms

- Agreement is a **safety** property.
 - Every possible state of the system has this property in all possible executions.
 - I.e., either they have not agreed yet, or they all agreed on the same value.
- Termination is a **liveness** property.
 - Some state of the system has this property in all possible executions.
 - The property is stable: once some state of an execution has the property, all subsequent states also have it.

Consensus for three processes



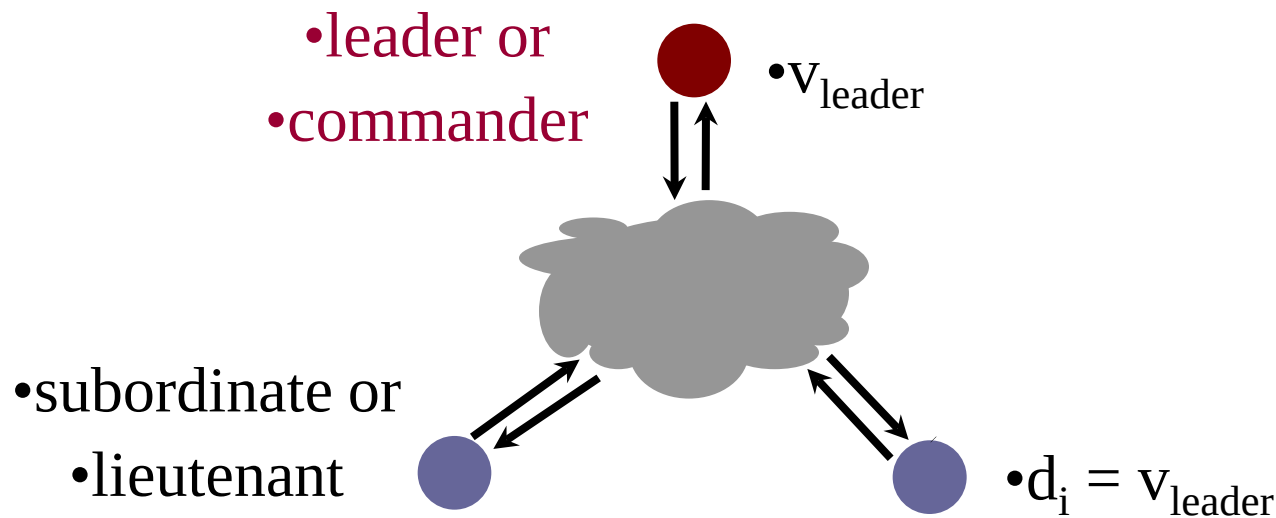
Variant I: Consensus (C)



$$\bullet d_i = v_k$$

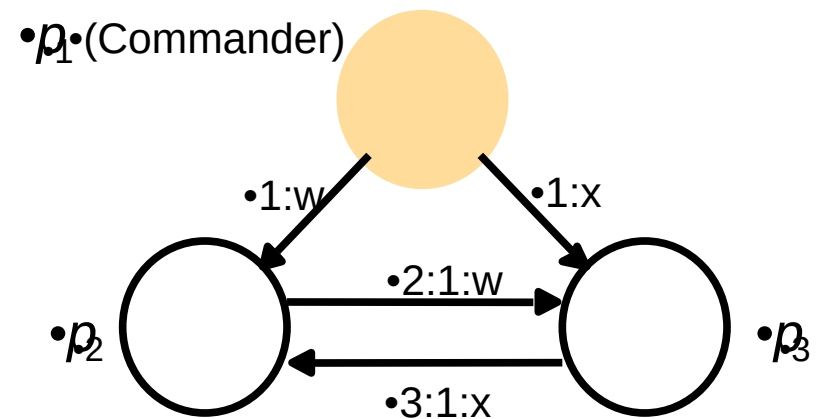
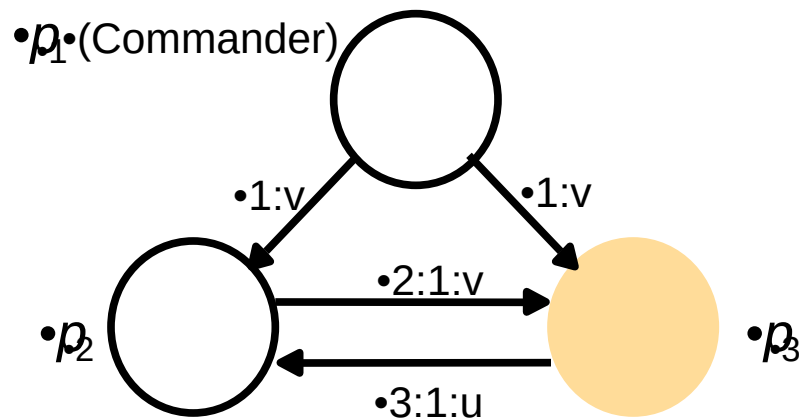
- P_i selects d_i from $\{v_0, \dots, v_{N-1}\}$.
- All P_i select d_i as the same v_k .
- If all P_i propose the same v , then $d_i = v$, else d_i is arbitrary.

Variant II: Command Consensus (BG)



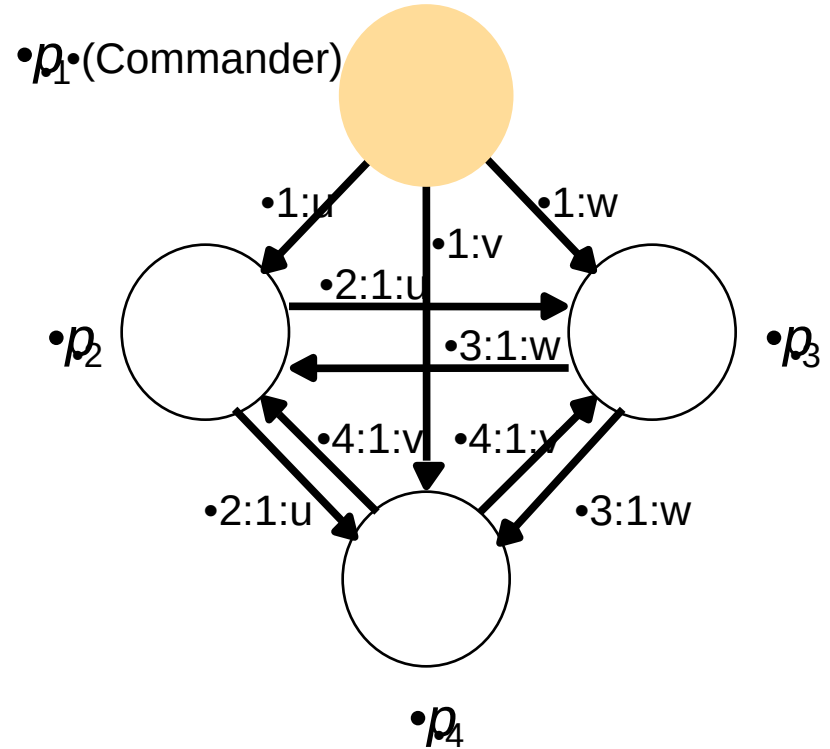
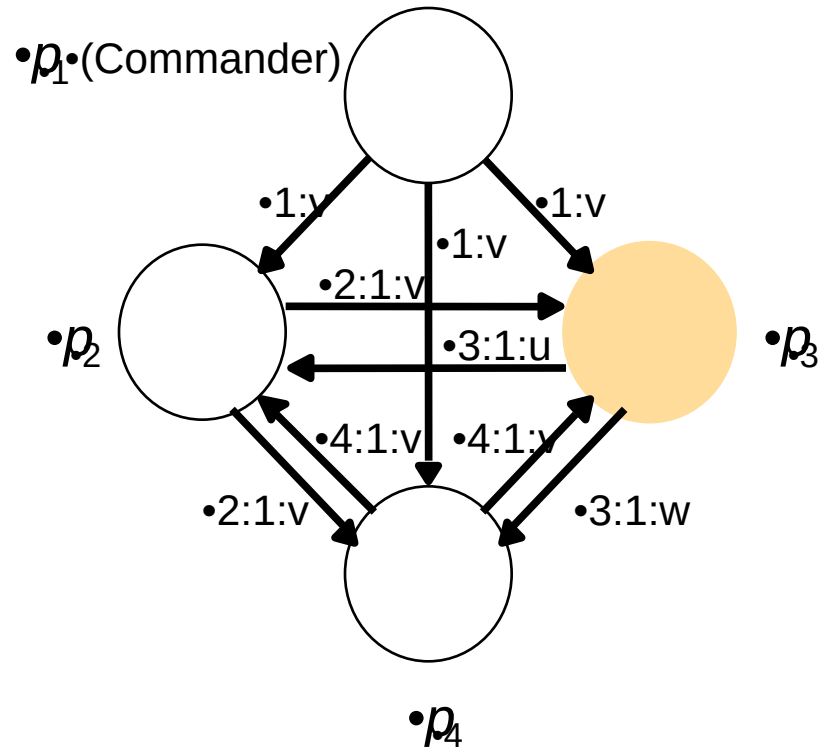
- P_i selects $d_i = v_{\text{leader}}$ proposed by designated **leader** node P_{leader} if the leader is correct, else the selected value is arbitrary.
- As used in the *Byzantine generals* problem.
- Also called *attacking armies*.

Three Byzantine generals



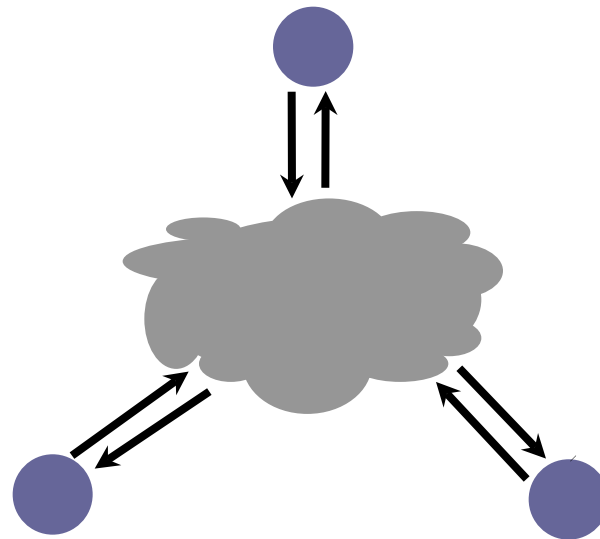
• Faulty processes are shown coloured

Four Byzantine generals



• Faulty processes are shown coloured

Variant III: Interactive Consistency (IC)



$$\bullet d_i = [v_0, \dots, v_{N-1}]$$

- P_i selects $d_i = [v_0, \dots, v_{N-1}]$ vector reflecting the values proposed by all correct participants.

Equivalence of Consensus Variants

- If any of the consensus variants has a solution, then all of them have a solution.
- Proof is by reduction.
 - IC from BG. Run BG N times, one with each P_i as leader.
 - C from IC. Run IC, then select from the vector.
 - BG from C.
 - Step 1: leader proposes to all subordinates.
 - Step 2: subordinates run C to agree on the proposed value.
 - IC from C? BG from IC? Etc.

Fischer-Lynch-Patterson (1985)

- No consensus can be **guaranteed** in an asynchronous communication system in the presence of any failures.
- Intuition: a “failed” process may just be slow, and can rise from the dead at exactly the wrong time.
- Consensus **may** occur recognizably, rarely or often.
 - e.g., if no inconveniently delayed messages
- FLP implies that no agreement can be guaranteed in an asynchronous system with Byzantine failures either. (More on that later.)

Consensus in Practice I

- What do these results mean in an asynchronous world?
 - Unfortunately, the Internet is asynchronous, even if we believe that all faults are eventually repaired.
 - Synchronized clocks and predictable execution times don't change this essential fact.
- Even a single faulty process can prevent consensus.
- Consensus is a practical necessity, so what are we to do?

Consensus in Practice II

- We can use some tricks to apply synchronous algorithms:
 - Fault masking: assume that failed processes always recover, and reintegrate them into the group.
 - If you haven't heard from a process, wait longer...
 - A round terminates when every expected message is received.
 - Failure detectors: construct a failure detector that can determine if a process has failed.
 - Use a failure detector that is live but not accurate.
 - Assume bounded delay.
 - Declare slow nodes dead and fence them off.
- But: protocols may block in pathological scenarios, and they may misbehave if a failure detector is wrong.

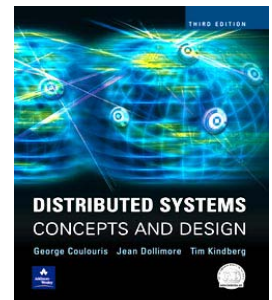
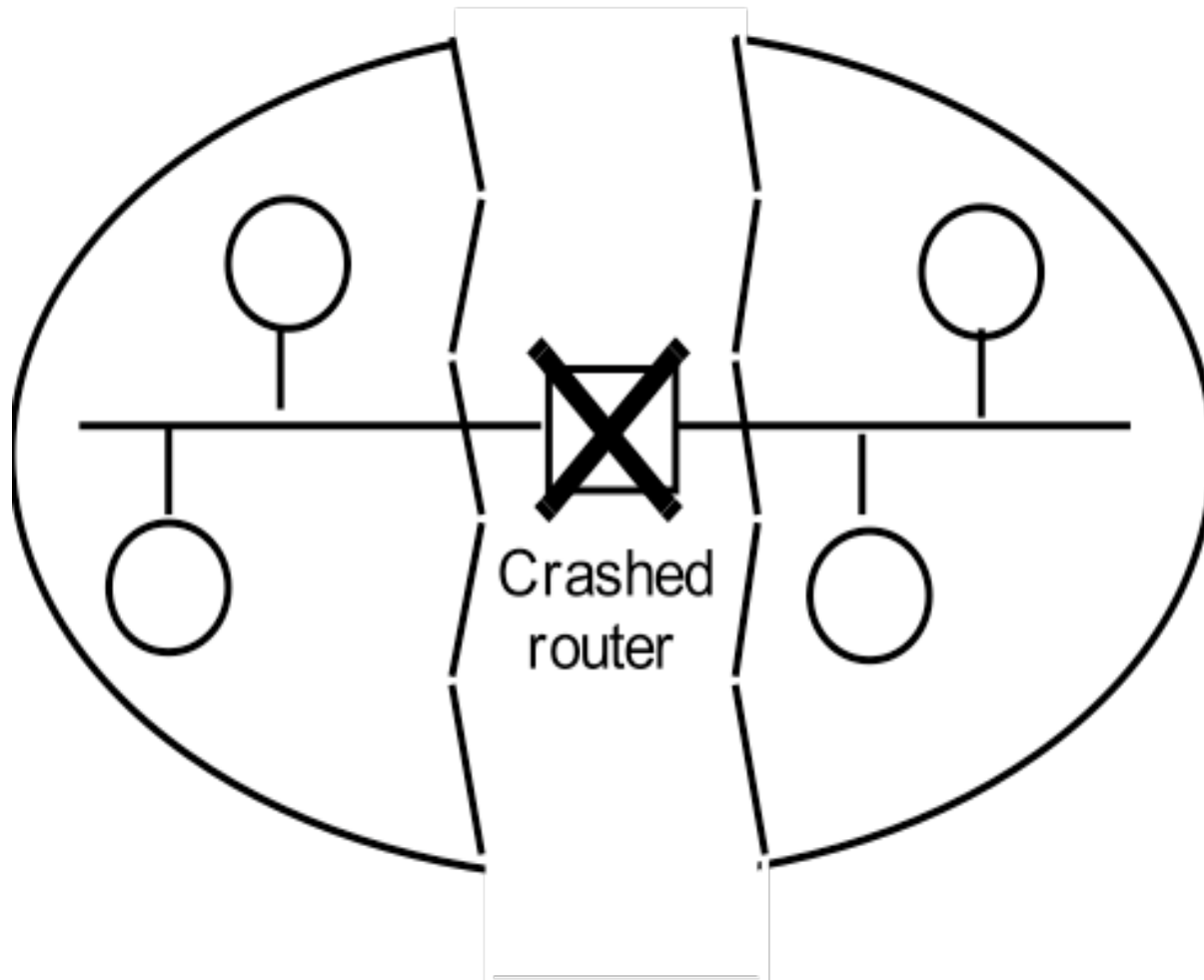
Failure Detectors

- How to detect that a member has failed?
 - pings, timeouts, beacons, heartbeats
 - recovery notifications
 - “I was gone for awhile, but now I’m back.”
- Is the failure detector accurate?
- Is the failure detector live (complete)?
- In an asynchronous system, it is possible for a failure detector to be accurate or live, but not both.
 - FLP tells us that it is impossible for an asynchronous system to agree on anything with accuracy and liveness!

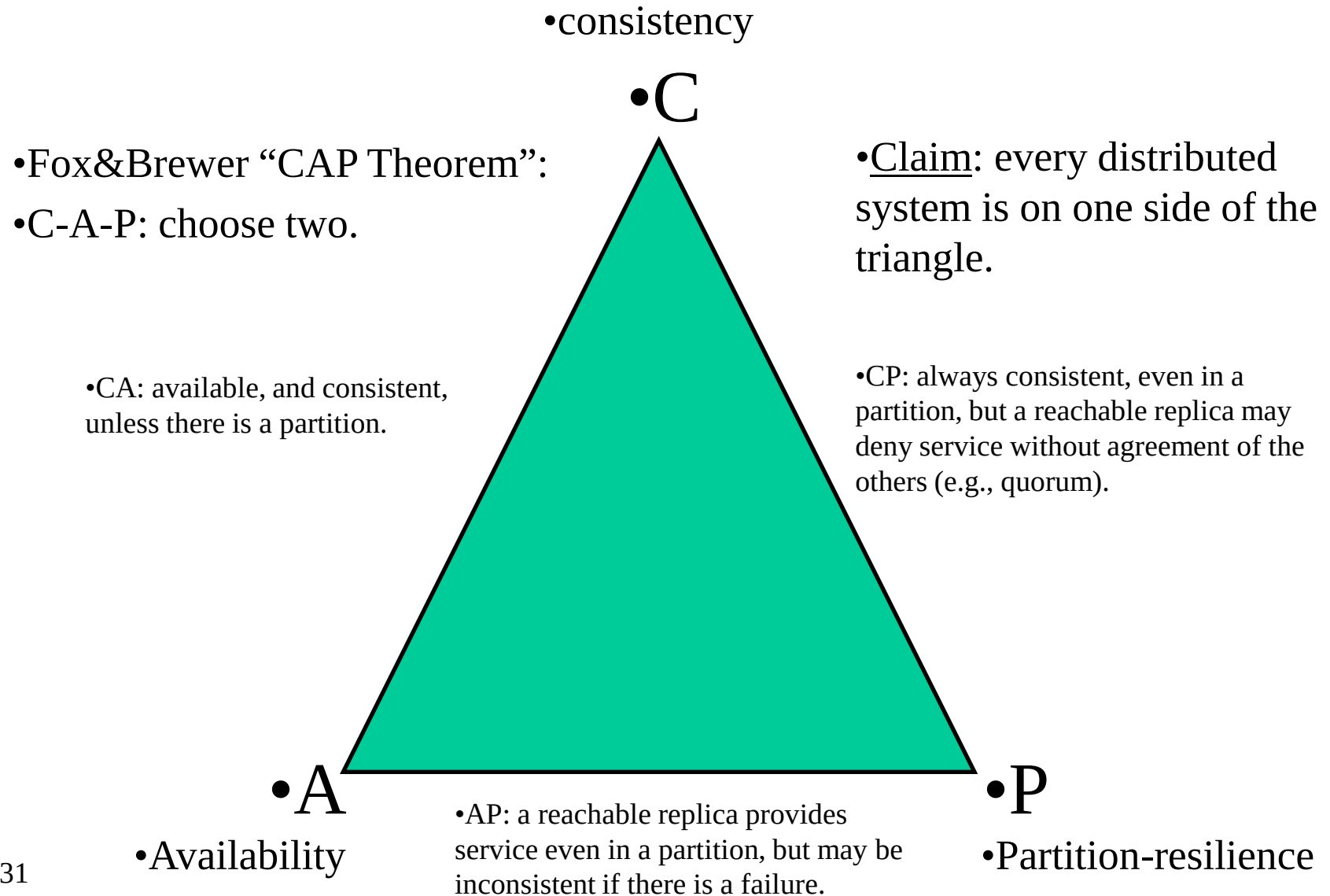
Failure Detectors in Real Systems

- Use a detector that is live but not accurate.
 - Assume bounded processing delays and delivery times.
 - Timeout with multiple retries detects failure accurately with high probability. Tune it to observed latencies.
 - If a “failed” site turns out to be alive, then restore it or kill it (fencing, fail-silent).
 - Example: leases and leased locks
- What do we assume about communication failures? How much pinging is enough? What about network partitions?

A network partition



CAP Theorem: Brewer



What is consistency?

- Consistency model:
 - A constraint on the system state observable by application operations
- Examples:
 - X86 memory:

•write x=5 •read x (should be 5)

– Database:

•x:=x+1; y:=y •assert(x+y==const)

•time

•time

Consistency

- No right or wrong consistency models
 - Tradeoff between ease of programmability and efficiency
- Consistency is hard in (distributed) systems:
 - Data replication (caching)
 - Concurrency
 - Failures

Strict consistency

- Each operation is stamped with a global wall-clock time
- Rules:
 1. Each read gets the latest write value
 2. All operations at one CPU have time-stamps in execution order

Strict consistency gives “intuitive” results

- No two CPUs in the critical section
- Proof: suppose mutual exclusion is violated

CPU0: $W(x)1$ $R(y)0$

CPU1: $W(y)1$ $R(x)0$

• W must have timestamp later than R

- Rule 1: read gets latest write

CPU0: $W(x)1$ $R(x)0$

CPU1:

$W(y)1$ $R(x)0$

• Contradicts rule 1: R must see $W(x)1$

Sequential consistency

- Strict consistency is not practical
 - No global wall-clock available
- Sequential consistency is the closest
- Rules: There is a **total order** of ops s.t.
 - All CPUs see results according to total order (i.e. reads see most recent writes)
 - Each CPUs' ops appear in order

Sequential Consistency

- A data store is sequentially consistent when:
- The result of any execution is the same as if the (read and write) operations by all processes on the data store ...
- were executed in some sequential order and ...
- the operations of each individual process appear ...
 - in this sequence
 - in the order specified by its program.

Sequential Consistency (3)

- (a) A sequentially consistent data store.
- (b) A data store that is not sequentially consistent.

P1: W(x)a

P2: W(x)b

P3: R(x)b R(x)a

P4: R(x)b R(x)a

(a)

P1: W(x)a

P2: W(x)b

P3: R(x)b R(x)a

P4: R(x)a R(x)b

(b)

Lamport clock gives a total order

- Each CPU keeps a logical clock
- Each CPU updates its logical clock between successive events
- A sender includes its clock value in the message.
- A receiver advances its clock to be greater than the message's clock value.
- Lamport clocks define a total order.
 - Ties are broken based on CPU ids.

Beyond Lamport clock

- Typical system obtains a total order differently
 - Use a single node to order all reads/writes
 - E.g. the lock_server in Lab1
 - Partition state over multiple nodes, each node orders reads/writes for its partition
- Invariant: exactly one is in charge of ordering
- ✗ The ordering node must be online

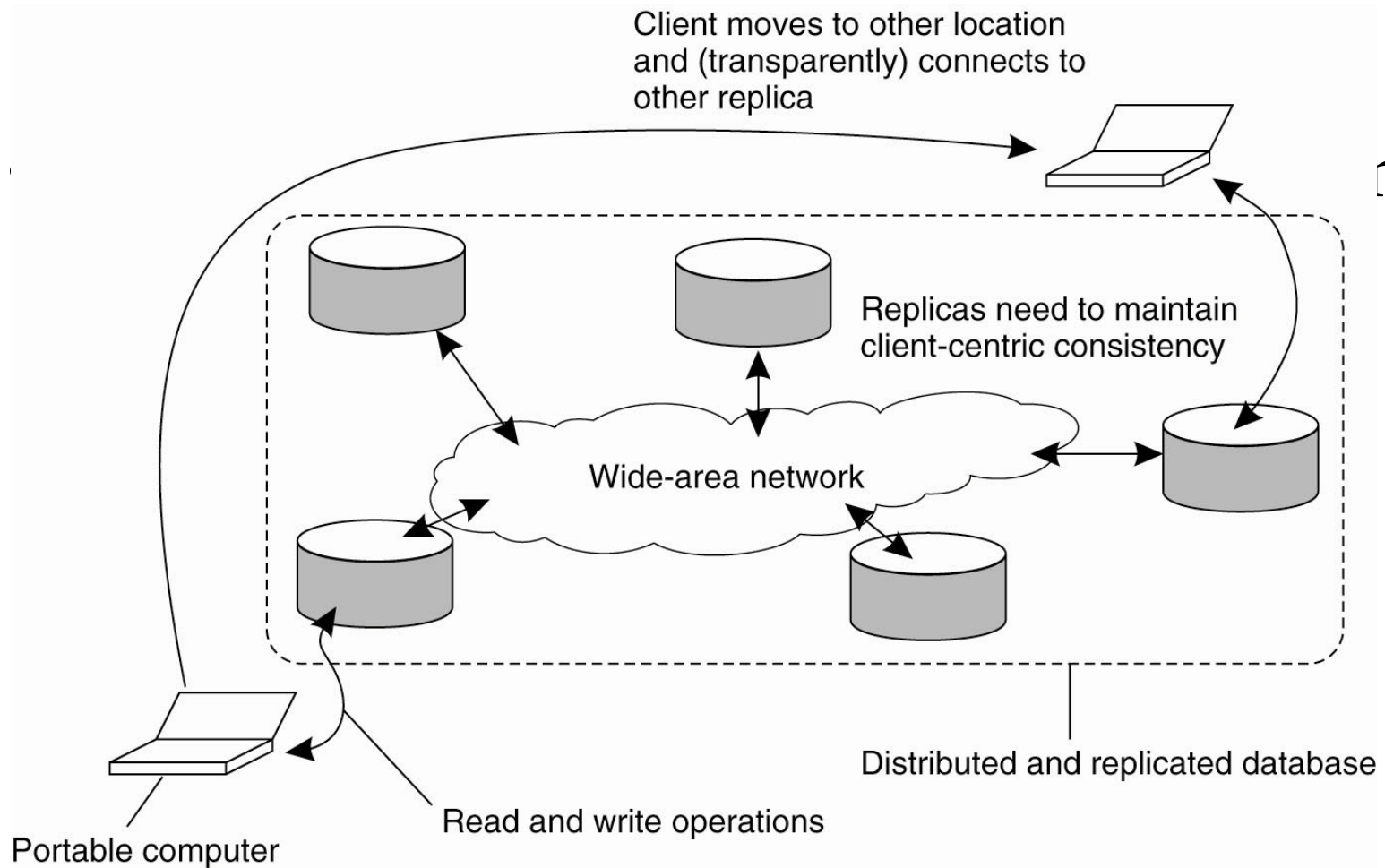
Weakly consistent systems

- Sequential consistency
 - All read/writes are applied in total order
 - Reads must see most recent writes
- Eventual consistency (Bayou)
 - Writes are *eventually* applied in total order
 - Reads might not see most recent writes in total order

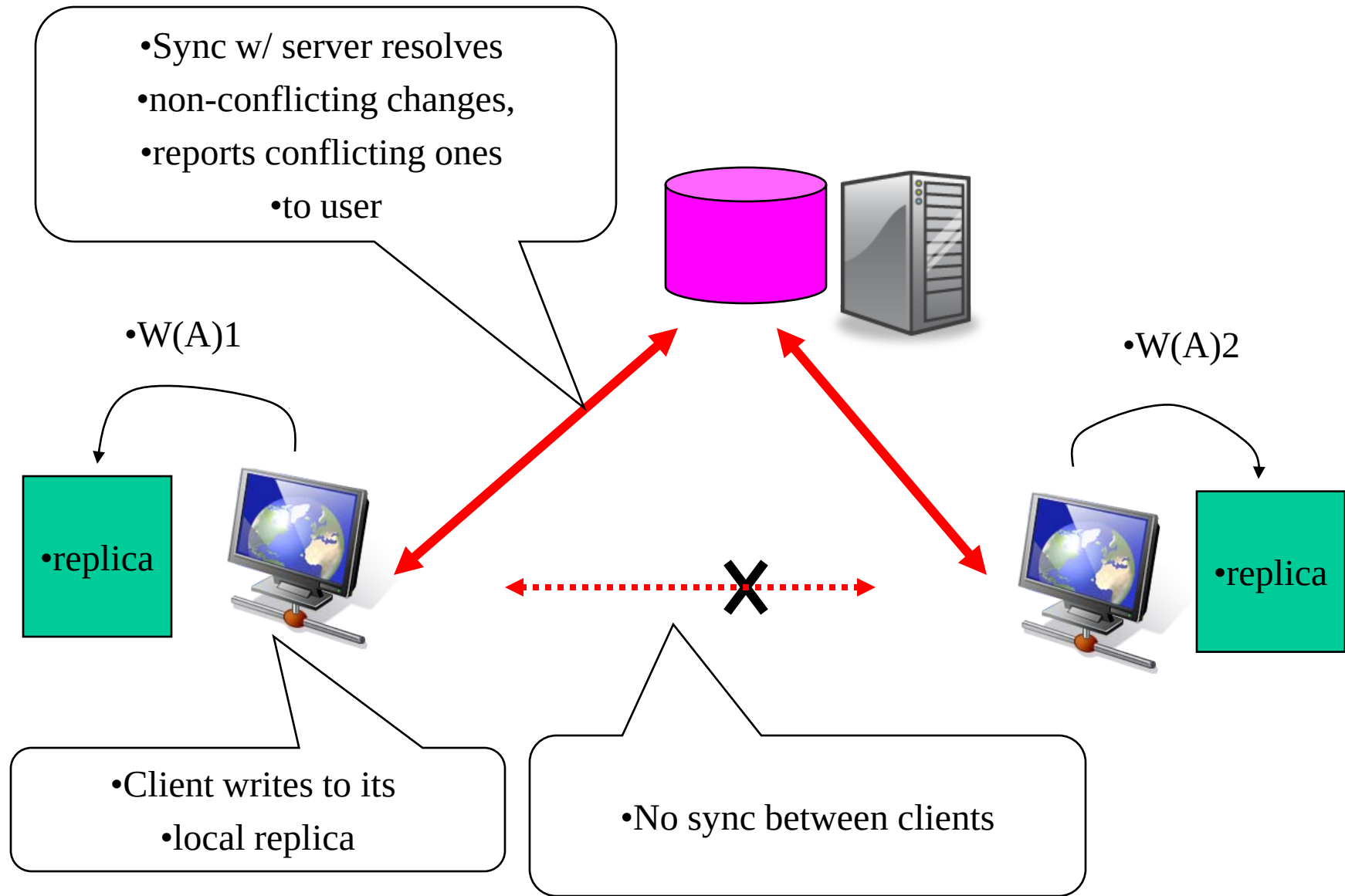
Why (not) eventual consistency?

- Support disconnected operations
 - Better to read a stale value than nothing
 - Better to save writes somewhere than nothing
- Potentially anomalous application behavior
 - Stale reads and conflicting writes...

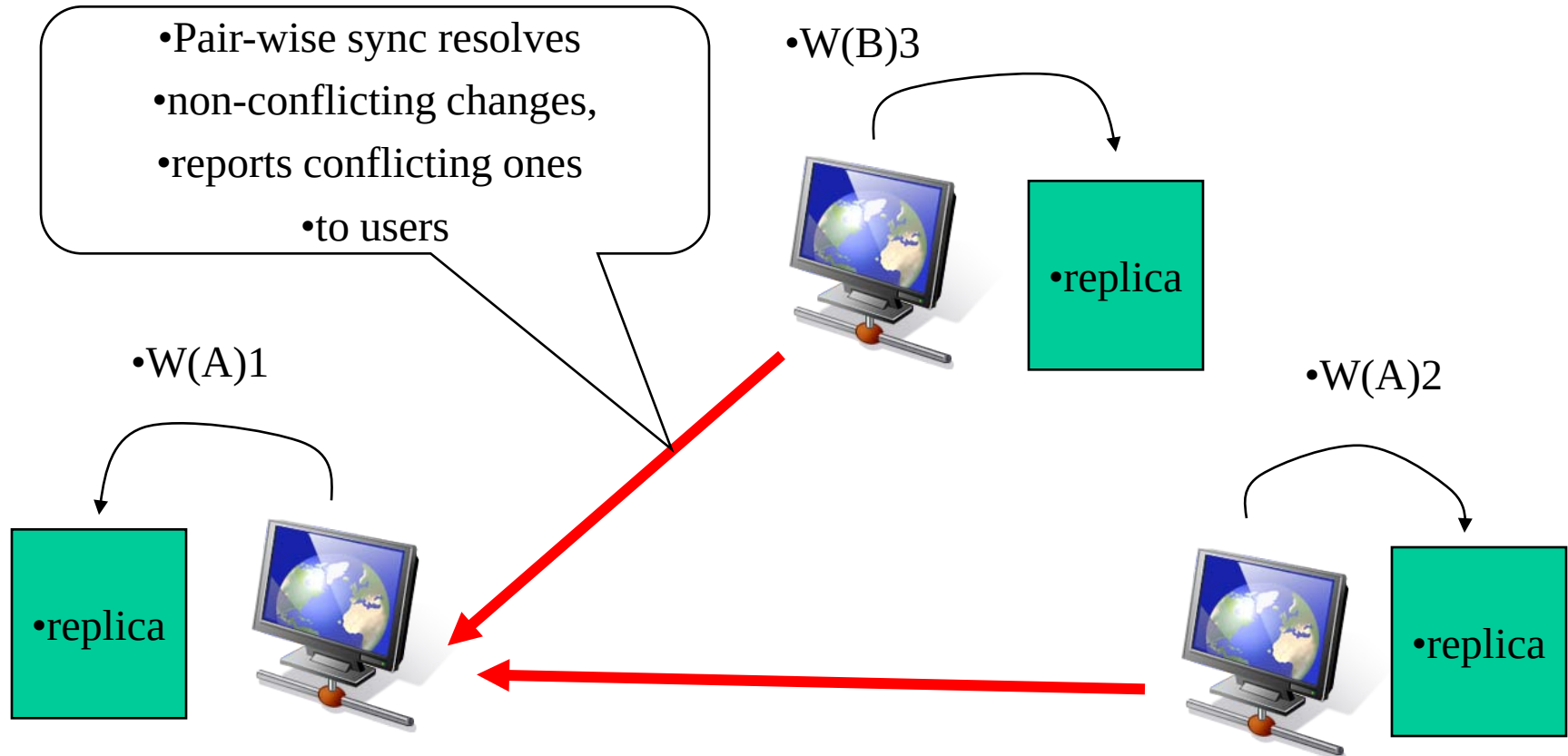
Eventual Consistency



Operating w/o total connectivity



Pair-wise synchronization



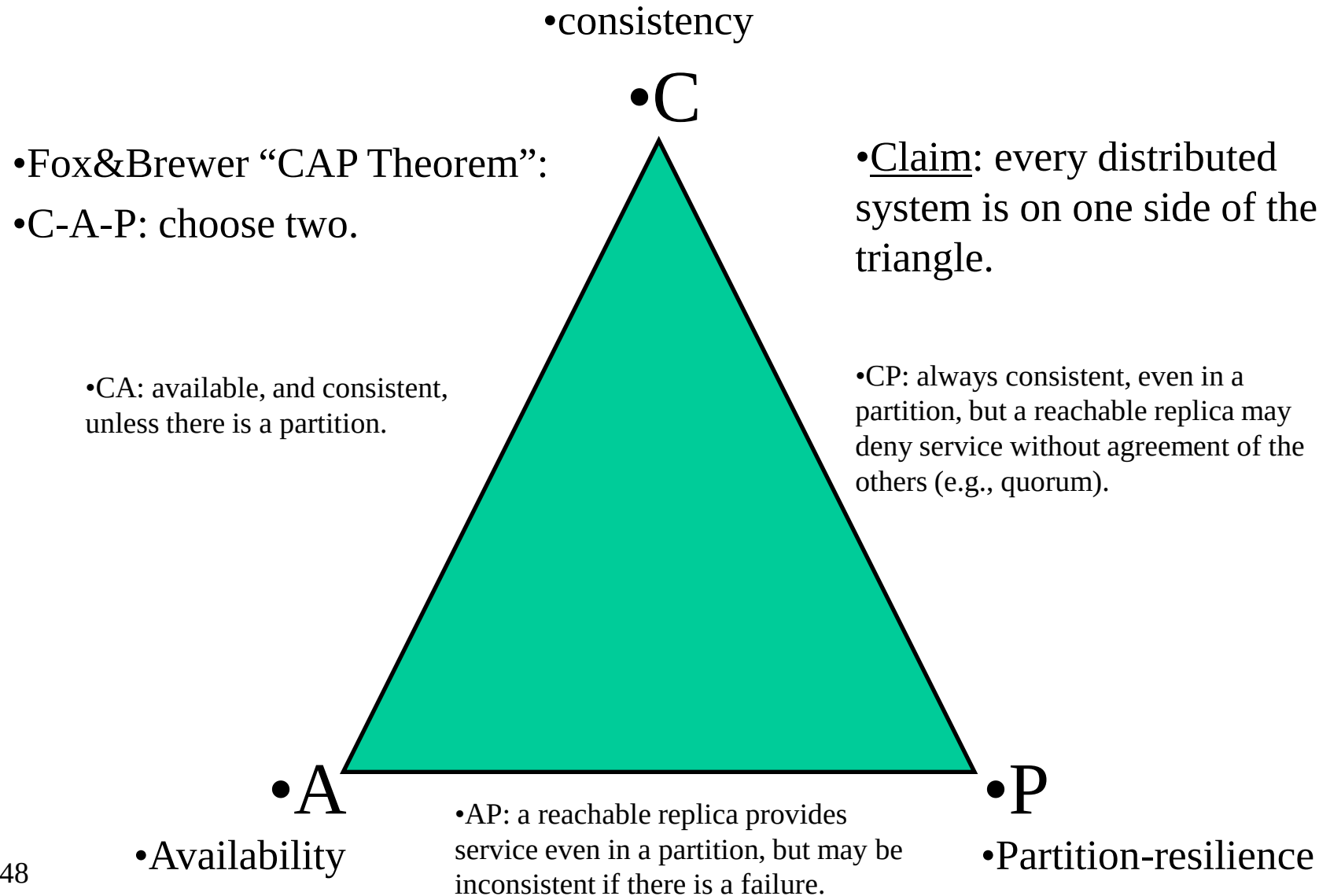
Prevent lost updates

- Detect if updates were sequential
 - If so, replace old version with new one
 - If not, detect conflict
- “Optimistic” vs. “Pessimistic”
 - Eventual Consistency: Let updates happen, worry about whether they can be serialized later
 - Sequential Consistency: Updates cannot take effect unless they are serialized first

Examples

- Strict consistency
 - ACID Databases (**A**tomicity, **C**onsistency, **I**solation, **D**urability)
 - Transactional systems
 - 2PC, Paxos, Chubby
- Eventual consistency
 - BASE Databases (**B**asically **A**vailable, **S**oft state, **E**ventual consistency)
 - Amazon Dynamo
 - NoSQL, Key Value Stores

CAP Theorem: Brewer



Transactions in Distributed systems

- A sequence of operations that perform a single logical function
 - Separate from all other transactions
- Examples
 - Withdrawing money from your account
 - Making an airline reservation
 - Making a credit-card purchase
 - Registering for a course at WPI
 - ...
- Usually used in context of databases

Definition – *Atomic Transaction*

- A transaction that happens completely or not at all
 - No partial results
- Example:
 - Cash machine hands you cash and deducts amount from your account
 - Airline confirms your reservation and
 - Reduces number of free seats
 - Charges your credit card
 - (Sometimes) increases number of meals loaded onto flight
 - ...

Atomic Transaction Review

- Fundamental principles – *A C I D*
 - *Atomicity* – to outside world, transaction happens indivisibly
 - *Consistency* – transaction preserves system invariants
 - *Isolated* – transactions do not interfere with each other
 - *Durable* – once a transaction “commits,” the changes are permanent

Programming in a Transaction System

- *Begin_transaction*
 - Mark the start of a transaction
- *End_transaction*
 - Mark the end of a transaction and try to “commit”
- *Abort_transaction*
 - Terminate the transaction and restore old values
- *Read*
 - Read data from a file, table, etc., on behalf of the transaction
- *Write*
 - Write data to file, table, etc., on behalf of the transaction

Programming in a Transaction System

(continued)

- As a matter of practice, separate transactions are handled in separate threads or processes
- *Isolated* property means that two concurrent transactions are *serialized*
 - With respect to any shared data
 - I.e., they run in some indeterminate order with respect to each other

Programming in a Transaction System

(continued)

- *Nested Transactions*
 - One or more transactions inside another transaction
 - May individually commit, *but* may need to be undone
- **Example**
 - Planning a trip involving three flights
 - Reservation for each flight “commits” individually
 - Must be undone if entire trip cannot commit

Tools for Implementing Atomic Transactions

(in each node of the distributed system)

- Stable storage
 - i.e., write to disk “atomically” ([ppt](#), [html](#) – slide #22)
- Log file
 - i.e., record actions in a log before “committing” them ([ppt](#), [html](#) – slide #26)
 - Log in stable storage
- Locking protocols
 - Serialize *Read* and *Write* operations of same data by separate transactions
- ...

Tools for Implementing Atomic Transactions

(continued)

- *Begin_transaction*
 - Place a *begin* entry in log
- *Write*
 - Write updated data to log
- *Abort_transaction*
 - Place *abort* entry in log
- *End_transaction* (i.e., *commit*)
 - Place *commit* entry in log
 - Copy logged data to files
 - Place *done* entry in log

Tools for Implementing Atomic Transactions

(continued)

- Crash recovery – search log
 - If *begin* entry, look for matching entries
 - If *done*, do nothing (all files have been updated)
 - If *abort*, undo any permanent changes that transaction may have made
 - If *commit* but not *done*, copy updated blocks from log to files, then add *done* entry

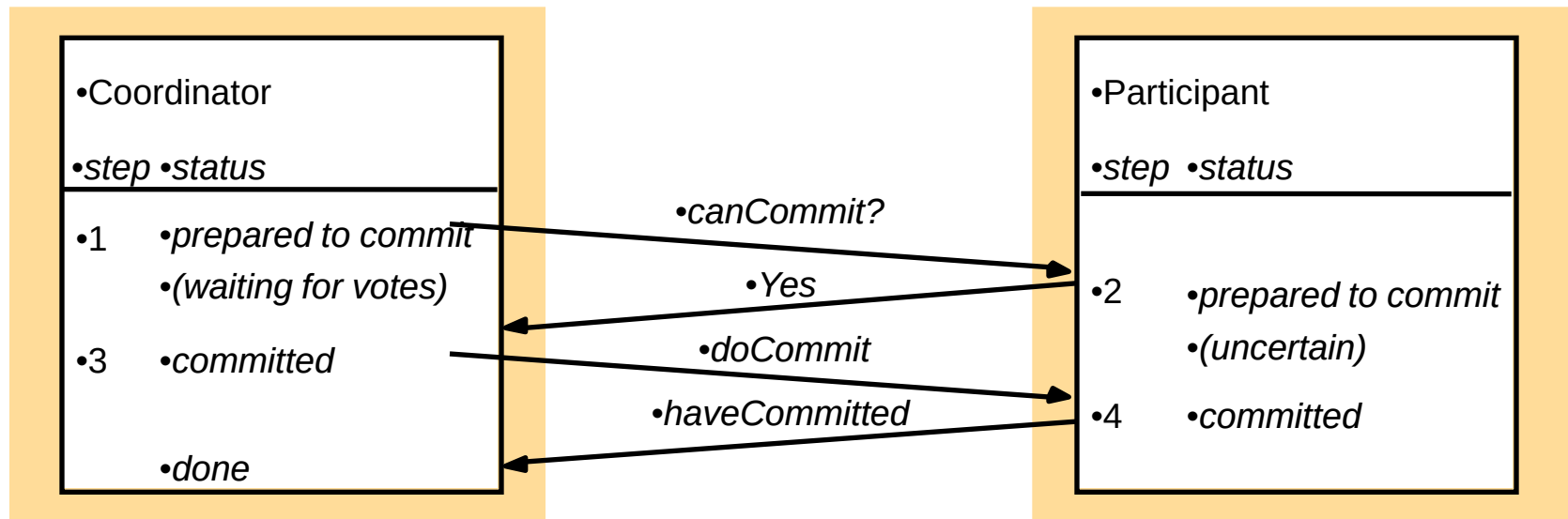
Distributed Atomic Transactions

- Atomic transactions that span multiple sites and/or systems
- Same semantics as atomic transactions on single system
 - *A C I D*
- Failure modes
 - Crash or other failure of one site or system
 - Network failure or partition
 - Byzantine failures

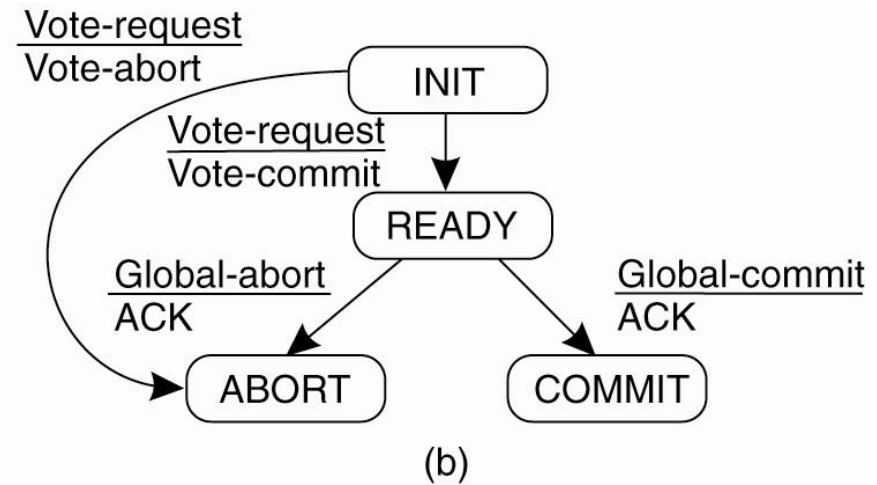
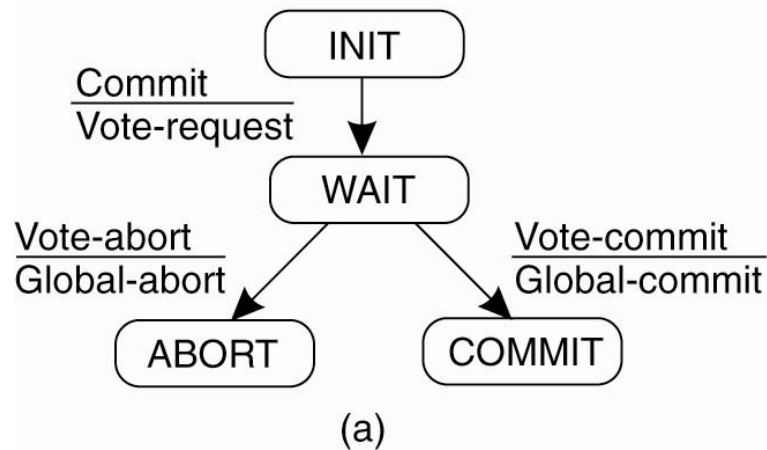
General Solution: The two-phase commit protocol [DΣIM]

- **Phase 1** (*voting phase*):
 1. The coordinator sends a *canCommit?* request to each of the participants in the transaction.
 2. When a participant receives a *canCommit?* request it replies with its vote (*Yes* or *No*) to the coordinator. Before voting *Yes*, it prepares to commit by saving objects in permanent storage. If the vote is *No* the participant aborts immediately.
- **Phase 2** (*completion according to outcome of vote*):
 3. The coordinator collects the votes (including its own).
 - (a) If there are no failures and all the votes are *Yes* the coordinator decides to commit the transaction and sends a *doCommit* request to each of the participants.
 - (b) Otherwise the coordinator decides to abort the transaction and sends *doAbort* requests to all participants that voted *Yes*.
 4. Participants that voted *Yes* are waiting for a *doCommit* or *doAbort* request from the coordinator. When a participant receives one of these messages it acts accordingly and in the case of commit, makes a *haveCommitted* call as confirmation to the coordinator.

Communication in two-phase commit protocol



Life Cycle of Two-Phase Commit (1)



- (a) The finite state machine for the coordinator in 2PC.
- (b) The finite state machine for a participant.

Two-Phase Commit (2)

- Actions taken by a participant P when residing in state READY and having contacted another participant Q.

State of Q	Action by P
COMMIT	Make transition to COMMIT
ABORT	Make transition to ABORT
INIT	Make transition to ABORT
READY	Contact another participant

Two-Phase Commit (3)

Actions by coordinator:

```
write START_2PC to local log;
multicast VOTE_REQUEST to all participants;
while not all votes have been collected {
    wait for any incoming vote;
    if timeout {
        write GLOBAL_ABORT to local log;
        multicast GLOBAL_ABORT to all participants;
        exit;
    }
    record vote;
}
```

Two-Phase Commit (4)

- Outline of the steps taken by the coordinator in a two-phase commit protocol.

```
if all participants sent VOTE_COMMIT and coordinator votes COMMIT {  
    write GLOBAL_COMMIT to local log;  
    multicast GLOBAL_COMMIT to all participants;  
} else {  
    write GLOBAL_ABORT to local log;  
    multicast GLOBAL_ABORT to all participants;  
}
```


Two-Phase Commit (5)

- (a) The steps taken by a participant process in 2PC.

actions by participant:

```
write INIT to local log;
wait for VOTE_REQUEST from coordinator;
if timeout {
    write VOTE_ABORT to local log;
    exit;
}
if participant votes COMMIT {
    write VOTE_COMMIT to local log;
    send VOTE_COMMIT to coordinator;
    wait for DECISION from coordinator;
    if timeout {
        multicast DECISION_REQUEST to other participants;
        wait until DECISION is received; /* remain blocked */
        write DECISION to local log;
    }
    if DECISION == GLOBAL_COMMIT
        write GLOBAL_COMMIT to local log;
    else if DECISION == GLOBAL_ABORT
        write GLOBAL_ABORT to local log;
} else {
    write VOTE_ABORT to local log;
    send VOTE_ABORT to coordinator;
}
```

(a)

Two-Phase Commit (7)

Actions for handling decision requests: /* executed by separate thread */

```
while true {  
    wait until any incoming DECISION_REQUEST is received; /* remain blocked */  
    read most recently recorded STATE from the local log;  
    if STATE == GLOBAL_COMMIT  
        send GLOBAL_COMMIT to requesting participant;  
    else if STATE == INIT or STATE == GLOBAL_ABORT  
        send GLOBAL_ABORT to requesting participant;  
    else  
        skip; /* participant remains blocked */  
}
```

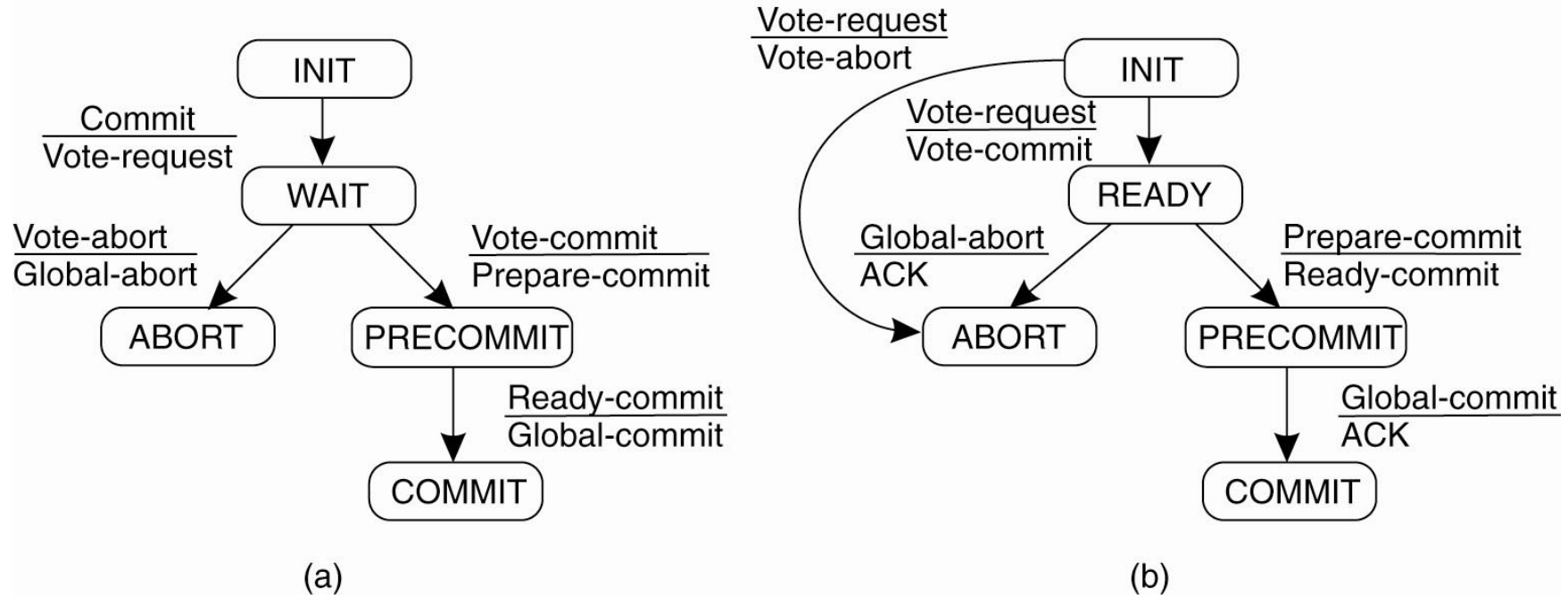
(b)

- (b) The steps for handling incoming decision requests..

Three-Phase Commit (1)

- The states of the coordinator and each participant
- satisfy the following two conditions:
 1. There is no single state from which it is possible to make a transition directly to either a COMMIT or an ABORT state.
 2. There is no state in which it is not possible to make a final decision, and from which a transition to a COMMIT state can be made.

Three-Phase Commit (2)



- (a) The finite state machine for the coordinator in 3PC. (b) The finite state machine for a participant.